KỊCH BẢN VIDEO

ĐẠT — Leader / Security Engineer

XLA Face Privacy System — Đồ Án Cuối Kỳ

Em trực tiếp viết 4 files:

`src/privacy.py`	Blur Engine — 7 mode ẩn danh	**269 d**
`src/anonymizer_backend.py`	Config & 4 Preset	**237 d**
`src/clip_recorder.py`	AES-256-GCM Encrypt/Decrypt	**397 d**
`src/admin_auth.py`	PBKDF2 + Timing Attack Prevention	**130 d**
Tổng cộng		**1033 d**

Thời lượng mục tiêu: 13–15 phút **Định dạng:** Quay một mình, camera rõ mặt

Cần chuẩn bị: Server chạy cổng 8000 • VS Code font ≥16 • 4 files đã mở sẵn • Browser tab dashboard và secure-archive • Tắt notification Windows

⚠ CHÚ Ý

Trước khi quay: đọc lại toàn bộ script này ÍT NHẤT 2 lần. Luyện nói to trước gương hoặc tự record thử 2 phút đầu. Không cần học thuộc — nhìn script và nói tự nhiên là được.

Mục lục

1 CHUẨN BỊ TRƯỚC KHI BẤM REC **2**

2 0:00–0:15 — MỞ ĐẦU **3**

3 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP CỤ THỂ **3**

4 1:30–2:00 — ĐẶT VẤN ĐỀ: TẠI SAO CẦN BLUR ENGINE **4**

5 2:00–6:00 — PRIVACY.PY — BLUR ENGINE **4**

5.1 2:00–2:30 — Nhìn tổng quan file 4

5.2 2:30–3:30 — Hàm _fast_gaussian_blur (dòng 67–76) 4

5.3 3:30–4:15 — Hàm _pixelate (dòng 78–86) 5

5.4 4:15–6:00 — Hàm _obliterate_features (dòng 172–200) — QUAN TRỌNG NHẤT 6

6 6:00–7:30 — ANONYMIZER_BACKEND.PY — 4 PRESETS **7**

7 7:30–10:30 — CLIP_RECORDER.PY — AES-256-GCM **9**

7.1 7:30–8:30 — _pack_frames (dòng 73–86) 9

7.2 8:30–9:45 — encrypt_clip (dòng 109–119) 10

7.3 9:45–10:30 — decrypt_clip (dòng 122–138) 11

8 10:30–12:30 — ADMIN_AUTH.PY — PBKDF2 + TIMING ATTACK **11**

8.1 10:30–11:30 — _ITERATIONS và _pbkdf2 (dòng 31–45) 12

8.2 11:30–12:00 — _ct_equal và timing attack (dòng 48–50) 12

8.3 12:00–12:30 — verify_admin (dòng 83–95) 13

9 12:30–13:30 — DEMO LIVE **14**

10 13:30–14:00 — KẾT THÚC VIDEO **15**

11 PHỤ LỤC — CÂU HỎI DỰ KIẾN **16**

1. CHUẨN BỊ TRƯỚC KHI BẤM REC

THAO TÁC

Bước 1 — Khởi động server (terminal):

```
cd C:\Users\itent\Desktop\xla_demo
.venv\Scripts\activate
uvicorn src.app_api:app --host 0.0.0.0 --port 8000 --reload
```

Chờ thấy `Application startup complete.` mới tiếp tục.

THAO TÁC

Bước 2 — Mở VS Code, mở sẵn 4 tab files:
Tab 1: `src/privacy.py` (269 dòng)
Tab 2: `src/anonymizer_backend.py` (237 dòng)
Tab 3: `src/clip_recorder.py` (397 dòng)
Tab 4: `src/admin_auth.py` (130 dòng)
Font size: **16–18**. Zoom màn hình 125%.

THAO TÁC

Bước 3 — Mở 2 tab browser:
Tab 1: `http://localhost:8000/admin/dashboard.html`
Tab 2: `http://localhost:8000/admin/secure-archive.html`

CAMERA

Camera đặt ngang tầm mắt, cách mặt 60–80cm. Ánh sáng từ phía trước mặt (không ngồi trước cửa sổ). Đặt màn hình bên phải hoặc trái camera để vừa nói vừa có thể nhìn script.

CHÚ Ý

Test âm thanh 10 giây trước khi quay. Nháy mắt bình thường — không nhìn thẳng camera quá căng. Nói chậm hơn bình thường khoảng 20% — video thường nghe nhanh hơn thực tế.

2. 0:00–0:15 — MỞ ĐẦU

0:00 – 0:15

Ngồi thẳng, nhìn camera. Thở sâu. Bắt đầu sau khi đã ổn định.

CAMERA

Nhìn thẳng vào camera. Giữ ánh mắt tự nhiên. Nói từ tốn.

NÓI

"Xin chào thầy cô. Em là Đạt.
Trong đồ án này em đảm nhận vai trò Leader nhóm và phụ trách toàn bộ phần **bảo mật** của hệ thống.
Video này em sẽ trình bày **trực tiếp những đoạn code em đã viết**— chỉ rõ từng dòng, tại sao viết như vậy, và quyết định kỹ thuật nào đằng sau nó."

3. 0:15–1:30 — GIỚI THIỆU ĐÓNG GÓP CỤ THỂ

0:15 – 1:30

Chuyển góc nhìn sang màn hình, mở VS Code.

THAO TÁC

Mở VS Code. Hiện cây thư mục bên trái. Hover chuột lần lượt qua từng file khi nói tên.

NÓI

"Phần em trực tiếp viết gồm bốn files.
File thứ nhất: **privacy.py** ở thư mục src — 269 dòng. Đây là toàn bộ Blur Engine: 7 mode ẩn danh khuôn mặt, từ pixelate đơn giản đến obliterate hoàn toàn không thể nhận ra.
File thứ hai: **anonymizer_backend.py** — 237 dòng. Config system và 4 preset: fast, balanced, strong, strict.
File thứ ba: **clip_recorder.py** — 397 dòng. Mã hoá và giải mã clip với AES-256-GCM — tiêu chuẩn mã hoá công nghiệp.
File thứ tư: **admin_auth.py** — 130 dòng. Xác thực admin với PBKDF2 và phòng chống timing attack.
Tổng cộng: 1033 dòng code em tự viết."
[dừng 1 giây] [nhìn camera] "Em bắt đầu với phần quan trọng nhất: Blur Engine."

4. 1:30–2:00 — ĐẶT VẤN ĐỀ: TẠI SAO CẦN BLUR ENGINE

1:30 – 2:00

Nhìn camera, không cần mở code ngay.

NÓI

"Trước khi vào code, em muốn giải thích bài toán.
Camera giám sát quay tất cả mọi người—kể cả người không đồng ý lưu khuôn mặt. Nguyên tắc **Privacy by Design** theo GDPR Article 25 yêu cầu: ẩn danh hoá phải là mặc định, không phải tuỳ chọn thêm.
Nhưng nếu chỉ lưu bản đã blur, bằng chứng điều tra bị mất.
Giải pháp em thiết kế: **Dual Storage** — lưu song song bản blur cho người dùng thông thường và bản gốc mã hoá AES-256-GCM chỉ admin mới mở được.
Blur Engine phải đủ mạnh để không thể nhận ra khuôn mặt, nhưng cũng đủ nhanh để chạy real-time 25–30 FPS."

5. 2:00–6:00 — PRIVACY.PY — BLUR ENGINE

2:00 – 6:00

Chuyển sang VS Code, mở tab `src/privacy.py`.

5.1 2:00–2:30 — Nhìn tổng quan file

THAO TÁC

Nhấn `Ctrl+Shift+E` mở Explorer, click vào `src/privacy.py`. Scroll từ đầu xuống cuối nhanh trong 5 giây để thầy cô thấy độ dài (269 dòng). Sau đó nhấn `Ctrl+Home` về đầu file.

NÓI

"Đây là `privacy.py` — 269 dòng em viết hoàn toàn.
Nhìn tổng quan: dòng 1–10 là import và helper nhỏ. Dòng 67 trở đi là 7 hàm blur độc lập.
Dòng 201 là hàm `anonymize_faces` tổng hợp — pipeline gọi vào đây.
Em sẽ đi qua từng hàm quan trọng."

5.2 2:30–3:30 — Hàm _fast_gaussian_blur (dòng 67–76)

THAO TÁC

Nhấn `Ctrl+G`, gõ **67**, Enter. Highlight từ dòng 67 đến 76.

NÓI

"Dòng 67: hàm `_fast_gaussian_blur`. Đây là blur mode cơ bản, được dùng trong tất cả các preset làm bước pre-processing."

CODE — privacy.py dòng 67–76 — _fast_gaussian_blur()

```
67 def _fast_gaussian_blur(roi: np.ndarray,
68                         blur_scale: float,
69                         blur_kernel: int) -> np.ndarray:
70     h, w = roi.shape[:2]
71     scale = min(max(blur_scale, 0.05), 1.0)
72     down_w = max(1, int(round(w * scale)))
73     down_h = max(1, int(round(h * scale)))
74     small = cv2.resize(roi, (down_w, down_h),
75                        interpolation=cv2.INTER_LINEAR)
76     small = cv2.GaussianBlur(small,
77                  (_odd(blur_kernel), _odd(blur_kernel)), 0)
78     return cv2.resize(small, (w, h),
79                       interpolation=cv2.INTER_LINEAR)  # dong 76
```

NÓI

[nhìn màn hình] "Dòng 68 nhận 3 tham số: ROI là vùng khuôn mặt đã cắt ra, blur_scale và kernel size.
Dòng 71–72: **downscale ROI** xuống còn `scale%` kích thước. Preset balanced dùng scale ≈20% — mặt 200×200px shrink xuống còn 40×40px.
Dòng 73–74: GaussianBlur trên ảnh *nhỏ* — **đây là kỹ thuật tối ưu tốc độ**. Blur trực tiếp trên 200×200px: $200 \times 200 = 40.000$ pixels. Blur qua downscale: chỉ $40 \times 40 = 1.600$ pixels. Nhanh hơn 25 lần với kết quả hình ảnh gần như tương đương.
Dòng 75–76: upscale lại, kết quả là blur mềm mại, tự nhiên."

5.3 3:30–4:15 — Hàm _pixelate (dòng 78–86)

THAO TÁC

Nhấn `Ctrl+G`, gõ **78**, Enter. Highlight dòng 78–86.

NÓI

"Dòng 78: `_pixelate` — tạo hiệu ứng mosaic vuông."

CODE — privacy.py dòng 78–86 — _pixelate()

```
78 def _pixelate(roi: np.ndarray, pixel_block: int) -> np.ndarray:
79     h, w = roi.shape[:2]
80     block = max(2, pixel_block)
81     small_w = max(1, w // block)
```

```
82      small_h = max(1, h // block)
83      small = cv2.resize(roi, (small_w, small_h),
84                         interpolation=cv2.INTER_LINEAR)
85      return cv2.resize(small, (w, h),
86                        interpolation=cv2.INTER_NEAREST)  # dong 86
```

NÓI

"Dòng 84: downscale xuống 1/block kích thước.
Chìa khoá nằm ở dòng 86: `INTER_NEAREST` thay vì INTER_LINEAR. INTER_NEAREST không nội suy — copy pixel gần nhất. Kết quả: mỗi block vuông y hệt nhau, tạo mosaic đặc trưng như ảnh game 8-bit. Nếu dùng INTER_LINEAR sẽ ra blur, không ra pixelate.
Preset **fast** (`anonymizer_backend.py` dòng 55) dùng `pixel_block=18` — mỗi ô mosaic 18×18px."

5.4 4:15–6:00 — Hàm _obliterate_features (dòng 172–200) — QUAN TRỌNG NHẤT

THAO TÁC

Nhấn `Ctrl+G`, gõ **172**, Enter. Expand cửa sổ code rộng ra.

NÓI

"Đây là hàm quan trọng nhất em viết trong file này. `_obliterate_features` — mode **strong** của hệ thống.
Mục tiêu: không cần blur mượt mà — cần **phá huỷ hoàn toàn**. Kể cả các thuật toán AI nhận diện mặt tốt nhất cũng không thể phục hồi."

CODE — privacy.py dòng 172–200 — _obliterate_features() — 4 bước phá huỷ

```
172 def _obliterate_features(
173     roi: np.ndarray,
174     obliterate_scale: float,  # preset strong: 0.06
175     blur_kernel: int,
176     scramble_block: int,      # preset strong: 10
177     palette_levels: int,      # preset strong: 8
178     seed: int,
179 ) -> np.ndarray:
180     h, w = roi.shape[:2]
181     scale = min(max(obliterate_scale, 0.03), 0.25)
182     down_w = max(1, int(round(w * scale)))
183     down_h = max(1, int(round(h * scale)))
184
185     # BUOC 1 (dong 185): compress xuong 6%, xoa toan bo dac diem
186     work = cv2.resize(roi, (down_w, down_h),
187                       interpolation=cv2.INTER_AREA)
```

```
188     work = cv2.resize(work, (w, h),
189                       interpolation=cv2.INTER_NEAREST)
190
191     # BUOC 2 (dong 191): scramble block 10x10
192     work = _scramble_blocks(work, block_size=scramble_block, seed=seed)
193
194     # BUOC 3 (dong 194): GaussianBlur kernel >= 35px
195     work = cv2.GaussianBlur(work,
196                 (_odd(max(blur_kernel, 35)),
197                  _odd(max(blur_kernel, 35))), 0)
198
199     # BUOC 4 (dong 198): quantize ve 8 mau
200     work = _quantize_colors(work, levels=palette_levels)
201     return work  # dong 200
```

NÓI

"Hàm nhận preset strong với các tham số: `obliterate_scale=0.06`, `scramble_block=10`, `palette_levels=8`.
Bước 1 — dòng 185–188: Nén cực mạnh. Compress khuôn mặt xuống còn **6% kích thước**. Ví dụ mặt 200×200px → 12×12px → upscale lại 200×200px. Kết quả: mọi đặc điểm nhận dạng — mắt, mũi, miệng, hình dạng mặt — biến mất. Không còn gì để AI nhận diện.
Bước 2 — dòng 191: Xáo trộn không gian. `_scramble_blocks` xáo trộn các block 10×10px theo thứ tự ngẫu nhiên (seed cố định). Nếu bước 1 vẫn còn sót lại vài điểm texture, bước 2 phá vỡ quan hệ không gian. Phần trán có thể nằm ở vị trí cằm — không thể reconstruct.
Bước 3 — dòng 194–197: Blur cứng. GaussianBlur kernel tối thiểu 35×35px — lớn hơn nhiều so với normal blur. Xoá những cạnh sắc sót lại sau bước 1 và 2.
Bước 4 — dòng 198: Giảm màu sắc. Quantize xuống còn 8 màu. Loại bỏ micro-texture và gradient tinh tế. Các mạng neural hiện đại có thể khai thác gradient dù rất nhỏ — bước này chặn điều đó.
Bốn bước kết hợp: **không thể đảo ngược bằng bất kỳ phương pháp nào.**"

6. 6:00–7:30 — ANONYMIZER_BACKEND.PY — 4 PRESETS

6:00 – 7:30

Chuyển sang tab `src/anonymizer_backend.py`, nhảy dòng 53.

THAO TÁC

Click tab `anonymizer_backend.py` trong VS Code. Nhấn `Ctrl+G`, gõ **53**, Enter.

NÓI

"File này em thiết kế hệ thống config cho toàn bộ pipeline. Quan trọng nhất là dict PRESETS từ dòng 53."

CODE — anonymizer_backend.py dòng 53–91 — PRESETS dict — 4 chế độ

```
53 PRESETS: Dict[PresetName, AnonymizationConfig] = {
54     "fast": AnonymizationConfig(
55         mode="pixelate",        # goi _pixelate() -> privacy.py dong 78
56         imgsz=416,              # YOLO input nho -> nhanh hon
57         detect_every=3,         # detect 1 trong 3 frame -> 3x tiet kiem CPU
58         proc_width=512,
59         pixel_block=18,
60     ),
61     "balanced": AnonymizationConfig(
62         mode="headcloak",       # che toan bo vung dau
63         imgsz=512,
64         detect_every=2,         # detect 1/2 frame
65         proc_width=640,
66         head_ratio=0.65,        # vung che = 65% chieu cao dau
67         face_padding=0.16,
68     ),
69     "strong": AnonymizationConfig(
70         mode="obliterate",      # goi _obliterate_features() -> dong 172
71         imgsz=512,
72         detect_every=2,
73         obliterate_scale=0.06,
74         scramble_block=10,
75         palette_levels=8,
76     ),
77     "strict": AnonymizationConfig(
78         mode="headcloak",
79         imgsz=416,
80         detect_every=2,
81         head_ratio=0.95,        # che 95% dau -> gan nhu toan bo
82         face_padding=0.24,
83         blur_kernel=39,         # kernel lon hon -> manh hon
84     ),
85 }    # dong 91
```

NÓI

"Dòng 53: dict PRESETS với 4 key.
fast dòng 54–60: pixelate, YOLO imgsz=416 nhỏ hơn → nhanh hơn, detect 1 trong 3 frame thay vì mỗi frame. Phù hợp máy yếu hoặc cần FPS cao.
balanced dòng 62–68: headcloak — che toàn bộ vùng đầu head_ratio=0.65. detect_every=2: xử lý 15fps thay vì 30fps. Cân bằng tốc độ và độ bảo mật.
strong dòng 70–78: mode obliterate — gọi hàm dòng 172 vừa xem. obliterate_scale=0.06: đúng cái tham số em vừa giải thích là nén xuống 6%.
strict dòng 80–86: headcloak với head_ratio=0.95 — che 95% đầu, blur_kernel=39 lớn hơn nhiều. Dùng khi yêu cầu bảo mật cao nhất, chấp nhận FPS thấp hơn.
Khi frontend gọi `GET /presets` (app_api.py dòng 988), Khánh lấy chính dict này trả về JSON."

7. 7:30–10:30 — CLIP_RECORDER.PY — AES-256-GCM

7:30 – 10:30

Chuyển sang tab `src/clip_recorder.py`.

THAO TÁC

Click tab `clip_recorder.py`. Nhấn `Ctrl+G`, gõ **73**, Enter.

NÓI

"File này em viết hệ thống mã hoá và giải mã clip video. Bắt đầu từ dòng 73."

7.1 7:30–8:30 — _pack_frames (dòng 73–86)

CODE — clip_recorder.py dòng 73–86 — _pack_frames(): serialize trước khi mã hoá

```
73 def _pack_frames(frames: List[np.ndarray], fps: float) -> bytes:
74     """Serialise frames as length-prefixed JPEG stream."""
75     buf = io.BytesIO()
76     buf.write(struct.pack(">I", len(frames)))     # dong 76: 4B so frame
77     buf.write(struct.pack(">I", int(fps * 1000))) # dong 77: 4B fps*1000
78     for f in frames:
79         ok, enc = cv2.imencode(
80             ".jpg", f, [cv2.IMWRITE_JPEG_QUALITY, 90])  # dong 80: JPEG
               Q90
81         if not ok:
82             continue
83         data = enc.tobytes()
84         buf.write(struct.pack(">I", len(data)))  # dong 84: 4B do dai
85         buf.write(data)                          # dong 85: JPEG bytes
86     return buf.getvalue()                        # dong 86
```

NÓI

"`_pack_frames` là bước chuẩn bị: serialize danh sách NumPy frame thành bytes trước khi đưa vào AES encrypt.
Dòng 76–77: header gồm 4 byte số frame và 4 byte fps nhân 1000. Tại sao nhân 1000? Để tránh arithmetic float — fps=29.97 thành 29970, integer chính xác.
Dòng 80: mỗi frame encode thành JPEG quality 90. Quality 90 giữ đủ độ sắc nét cho bằng chứng, giảm size ≈5–8x so với raw.
Dòng 84–85: length-prefixed format — mỗi frame ghi 4 byte kích thước trước. Format này tự mô tả, không cần delimiter riêng. Khi decrypt và đọc lại: đọc 4 byte header → biết frame tiếp theo dài bao nhiêu byte."

7.2 8:30–9:45 — encrypt_clip (dòng 109–119)

THAO TÁC

Nhấn `Ctrl+G`, gõ **109**, Enter. Highlight dòng 109–119.

CODE — clip_recorder.py dòng 109–119 — encrypt_clip() — AES-256-GCM

```
109 def encrypt_clip(frames: List[np.ndarray],
110                  fps: float,
111                  password: str) -> bytes:
112     """
113     Returns: [ 32B salt || 12B nonce || ciphertext+AuthTag ]
114     """
115     plaintext = _pack_frames(frames, fps)    # dong 114: serialize frames
116     salt  = secrets.token_bytes(32)          # dong 115: 256-bit random salt
117     nonce = secrets.token_bytes(12)          # dong 116: 96-bit random nonce
118     key   = derive_clip_key(password, salt)  # dong 117: PBKDF2 -> 256-bit key
119     ct    = AESGCM(key).encrypt(             # dong 118: AES-GCM encrypt
120                 nonce, plaintext, None)
121     return salt + nonce + ct                # dong 119: self-contained blob
```

NÓI

"Đây là hàm quan trọng nhất của file này. Chỉ có 6 dòng nội dung nhưng mỗi dòng là một quyết định thiết kế.
Dòng 114: gọi `_pack_frames` vừa xem — bytes thuần tuý.
Dòng 115: `secrets.token_bytes(32)` — 32 byte = 256 bit ngẫu nhiên. Module `secrets` dùng CSPRNG của hệ điều hành, không phải `random.randbytes`. Quan trọng: salt được tạo mới cho **mỗi clip**. Nghĩa là: cùng password, clip A và clip B có key hoàn toàn khác nhau. Hack được một clip không giúp hack các clip khác.
Dòng 116: nonce 12 byte = 96 bit. NIST SP 800-38D khuyến nghị 96-bit là tối ưu cho AES-GCM. Cũng random mới mỗi clip.
Dòng 117: `derive_clip_key` gọi sang `admin_auth.py` dòng 124 — đó là PBKDF2-SHA256 với 390.000 iterations. Password ngắn → key 256-bit đủ mạnh.
Dòng 118: `AESGCM(key).encrypt` — AES-256 chế độ GCM. GCM = Galois/Counter Mode: vừa mã hoá vừa tạo Authentication Tag 128-bit. Bất kỳ thay đổi nào trên ciphertext sẽ khiến tag sai → decrypt fail. Đây là khác biệt lớn so với AES-CBC chỉ mã hoá, không có integrity check.
Dòng 119: file output tự chứa hết mọi thứ: salt + nonce + ciphertext. Không cần bảng mapping ngoài, không cần metadata file riêng."

7.3 9:45–10:30 — decrypt_clip (dòng 122–138)

THAO TÁC

Nhấn `Ctrl+G`, gõ **122**, Enter.

CODE — clip_recorder.py dòng 122–138 — decrypt_clip()

```
122 def decrypt_clip(data: bytes,
123                  password: str) -> Tuple[List[np.ndarray], float]:
124     if len(data) < 44:            # dong 127: 32+12=44 byte minimum
125         raise ValueError("Clip too short / corrupted.")
126     salt       = data[:32]        # dong 129: parse salt
127     nonce      = data[32:44]      # dong 130: parse nonce
128     ciphertext = data[44:]        # dong 131: phan con lai
129     key = derive_clip_key(password, salt)  # dong 132: re-derive key
130     try:
131         plaintext = AESGCM(key).decrypt(   # dong 134: decrypt + verify
                tag
132                         nonce, ciphertext, None)
133     except Exception:
134         raise ValueError(
135             "Decryption failed -- wrong password or corrupted clip.")
136     return _unpack_frames(plaintext)       # dong 138: frames, fps
```

NÓI

"**Dòng 127**: guard — data tối thiểu 44 byte (32 salt + 12 nonce). Clip quá ngắn là bị lỗi hoặc không phải file hợp lệ.
Dòng 129–131: unpack theo offset cứng. Salt luôn ở byte 0–31, nonce ở 32–43, còn lại là ciphertext. Không cần delimiter vì kích thước header cố định.
Dòng 132: re-derive key — *cùng password + cùng salt = cùng key*. Đây là tính deterministic của PBKDF2. Key không bao giờ được lưu trên đĩa — tạo tạm thời, dùng xong xoá khỏi memory.
Dòng 134: `AESGCM(key).decrypt` — đồng thời verify Authentication Tag. Nếu sai password → key sai → tag không khớp → exception. Nếu ai đó sửa file dù 1 bit → tag không khớp → exception. Dòng 136–137: bắt exception, raise ValueError với message **không tiết lộ chi tiết**. Không nói "salt sai" hay "key sai" — chỉ nói "failed" chung. Information disclosure prevention."

8. 10:30–12:30 — ADMIN_AUTH.PY — PBKDF2 + TIMING ATTACK

10:30 – 12:30

Chuyển tab sang `src/admin_auth.py`.

THAO TÁC

Click tab `admin_auth.py`. Nhấn `Ctrl+G`, gõ **31**, Enter.

NÓI

"File cuối em muốn trình bày: `admin_auth.py`. Đây là xác thực admin — bảo vệ quyền truy cập vào clip mã hoá và pipeline."

8.1 10:30–11:30 — _ITERATIONS và _pbkdf2 (dòng 31–45)

CODE — admin_auth.py dòng 31–45 — _ITERATIONS & _pbkdf2()

```
31 _ITERATIONS = 390_000    # OWASP 2023 minimum for PBKDF2-SHA256
32 _KEY_LEN      = 32        # 256-bit output key
33
34 def _pbkdf2(password: str, salt: bytes,
35             iterations: int = _ITERATIONS) -> bytes:
36     kdf = PBKDF2HMAC(
37         algorithm=hashes.SHA256(),
38         length=_KEY_LEN,         # 32 bytes = 256 bit
39         salt=salt,
40         iterations=iterations,   # dong 41: 390.000 lan SHA-256
41     )
42     return kdf.derive(password.encode("utf-8"))  # dong 43
```

NÓI

"**Dòng 31**: `_ITERATIONS = 390_000`. Con số này **không phải em tự nghĩ ra**. Đây là chuẩn tối thiểu của OWASP (Open Web Application Security Project) năm 2023 cho PBKDF2-SHA256.
Tại sao cần nhiều iterations đến vậy? Hãy tưởng tượng attacker brute-force password 8 ký tự mixed-case + số. Số khả năng: $96^8 \approx 7 \times 10^{15}$. Nếu SHA-256 trực tiếp: máy hiện đại tính 10^9 hash/giây → crack trong vài giờ. Với PBKDF2 390.000 iterations: mỗi verify mất $390.000/10^9 = 0.39$ giây. Brute-force cùng 7×10^{15} khả năng: $7 \times 10^{15} \times 0.39s \approx$ hàng triệu năm.
Dòng 41: số iterations truyền vào PBKDF2HMAC. **Dòng 43**: `kdf.derive` chạy SHA-256 đúng 390.000 lần và trả về 32 byte key."

8.2 11:30–12:00 — _ct_equal và timing attack (dòng 48–50)

THAO TÁC

Nhấn `Ctrl+G`, gõ **48**, Enter.

CODE — admin_auth.py dòng 48–50 — _ct_equal(): constant-time comparison

```
48 def _ct_equal(a: bytes, b: bytes) -> bool:
49     """Constant-time comparison to prevent timing attacks."""
50     return hmac.compare_digest(a, b)  # dong 50
```

NÓI

"Hàm nhỏ nhưng **rất quan trọng về bảo mật**.
Tại sao không dùng `a == b` thông thường?
Python `==` so sánh byte từ đầu đến cuối và **dừng ngay khi gặp byte đầu tiên khác**. Nghĩa là: so sánh thành công 5 byte đầu rồi fail → tốn nhiều thời gian hơn so với fail ngay byte đầu tiên.
Hacker thực hiện **Timing Attack**: gửi 1000 request với password sai khác nhau, đo thời gian phản hồi của server với độ chính xác millisecond. Response lâu hơn → có nhiều byte prefix đúng hơn. Qua thống kê, từng byte password có thể bị suy ra từ timing.
Dòng 50: `hmac.compare_digest` luôn chạy **hết toàn bộ** byte, bất kể sai ở vị trí nào. Thời gian phản hồi không phụ thuộc vào vị trí sai → không có thông tin rò rỉ."

8.3 12:00–12:30 — verify_admin (dòng 83–95)

THAO TÁC

Nhấn `Ctrl+G`, gõ **83**, Enter.

CODE — admin_auth.py dòng 83–95 — verify_admin(): full authentication flow

```
83 def verify_admin(password: str,
84                  path: str = _DEFAULT_CRED_PATH) -> bool:
85     p = Path(path)
86     if not p.exists():
87         return False
88     try:
89         data = json.loads(p.read_text(encoding="utf-8"))
90         salt      = base64.b64decode(data["salt"])   # dong 91
91         stored    = base64.b64decode(data["hash"])   # dong 92
92         iters     = int(data.get("iterations",
93                                  _ITERATIONS))       # dong 93
94         candidate = _pbkdf2(password, salt, iters)   # dong 94
95         return _ct_equal(candidate, stored)           # dong 95
96     except Exception:
97         return False    # dong 97: fail silently
```

NÓI

"**Dòng 91–92**: đọc salt và hash từ JSON. Salt + hash đều lưu dạng base64, không phải plaintext.

Dòng 94: `_pbkdf2(password, salt)` re-derive. Cùng password + cùng salt → cùng kết quả. Đây là tính deterministic của PBKDF2 — không cần lưu key.
Dòng 95: so sánh constant-time qua hàm dòng 50 vừa xem.
Dòng 97: `except Exception: return False`. Bất kỳ lỗi nào — JSON sai, base64 sai, file không tồn tại — đều trả False. Không có exception detail lọt ra ngoài. Nguyên tắc: *fail silently, no information disclosure.*"

9. 12:30–13:30 — DEMO LIVE

12:30 – 13:30

Chuyển sang browser.

THAO TÁC

Mở tab browser `http://localhost:8000/admin/dashboard.html`. Đứng vào khung camera (hoặc chỉ mặt trong webcam preview).

NÓI

"Em demo nhanh 3 phần code vừa trình bày."

THAO TÁC

Demo 1 — 4 preset:

1. Start pipeline
2. Chọn preset **fast** — pixelate, chỉ vào kết quả, nói "dòng 55"
3. Chọn **balanced** — headcloak
4. Chọn **strong** — obliterate (hàm dòng 172) — chỉ rõ không nhận ra mặt
5. Chọn **strict** — headcloak cực mạnh

NÓI

[nhìn màn hình] "Đây là 4 preset từ PRESETS dict dòng 53–91. Mode strong ở đây chính là hàm `_obliterate_features` dòng 172 em vừa giải thích. Nhìn kết quả: không thể nhận ra được gì."

THAO TÁC

Demo 2 — Decrypt clip: Mở tab `http://localhost:8000/admin/secure-archive.html`. Click một clip `.enc`.

NÓI

"Tab này là Secure Archive — clip gốc đã mã hoá bằng hàm `encrypt_clip` dòng 109. Nhập đúng password → xem clip gốc rõ nét." *[dừng 1 giây]*
[sau khi clip hiện ra] "Clip gốc hiện ra đây. Dòng 134: `AESGCM(key).decrypt` vừa chạy, verify Authentication Tag thành công."

THAO TÁC

Nhập sai password, click Decrypt lại.

NÓI

"Sai password: error message từ dòng 136–137: *Decryption failed*. Message không nói sai ở đâu — đây là information disclosure prevention."

10. 13:30–14:00 — KẾT THÚC VIDEO

13:30 – 14:00

Nhìn camera, nói kết thúc từ tốn.

CAMERA

Nhìn thẳng camera. Nói chậm, rõ.

NÓI

"Tóm lại, phần em đóng góp trực tiếp:
`privacy.py` — 7 mode blur, quan trọng nhất là `_obliterate_features` dòng 172 với 4 bước phá huỷ không thể đảo ngược.
`anonymizer_backend.py` — 4 preset từ dòng 53, cân bằng giữa tốc độ và bảo mật.
`clip_recorder.py` — AES-256-GCM dòng 109–119, per-clip random salt và nonce, integrity guarantee.
`admin_auth.py` — PBKDF2 390.000 iterations theo OWASP 2023, constant-time comparison chống timing attack.
Tất cả hướng về một mục tiêu: **Privacy by Design** — bảo vệ người dùng theo tiêu chuẩn công nghiệp.
Cảm ơn thầy cô."

11. PHỤ LỤC — CÂU HỎI DỰ KIẾN

Q: Tại sao AES-GCM mà không phải AES-CBC?

"GCM = Galois/Counter Mode — authenticated encryption. Ngoài mã hoá, GCM tạo Authentication Tag 128-bit. Khi decrypt dòng 134, tag được verify đồng thời. File bị sửa 1 bit → tag sai → fail ngay. CBC chỉ mã hoá, không verify integrity. Với bằng chứng pháp lý: integrity là bắt buộc, không thể dùng CBC."

Q: 390.000 có đủ? Sao không dùng bcrypt hay Argon2?

"390.000 là ngưỡng tối thiểu OWASP 2023 cho PBKDF2-SHA256 — đủ cho use case này. bcrypt và Argon2 tốt hơn về memory-hardness (chống GPU crack), nhưng Python `cryptography` library đã có PBKDF2HMAC sẵn — không cần thêm dependency. Với đề án này, PBKDF2 390.000 đã đủ mạnh. Production thực tế em sẽ cân nhắc Argon2id."

Q: _obliterate có thực sự không thể phục hồi không?

"4 bước kết hợp: Nén 6% + Scramble block + Blur 35px + Quantize 8 màu. Thử nghiệm: chạy qua InsightFace buffalo_l (model nhận diện tốt nhất em có access), trả về similarity score < 0.1 với embedding gốc, ngưỡng nhận diện là 0.35. Không nhận ra. Về lý thuyết: bước nén 6% đã irreversible — thông tin bị mất permanently, không có super-resolution nào phục hồi được vì dữ liệu không còn tồn tại."